



POO - Unid I

Exercício 1:

Em um projeto de console criado com os modelos atuais do .NET, considere o arquivo Program.cs com apenas as linhas abaixo:

```
// Programa "Olá, Mundo" utilizando instruções de nível superior em C#  
Console.WriteLine("Olá, Mundo!");
```

Qual alternativa descreve corretamente o que precisa ser verdade para esse código compilar e o que o compilador produz como ponto de entrada do executável?

A)

A compilação exige using System; explícito, pois não existe suporte a usings implícitos; o ponto de entrada é resolvido em tempo de execução por reflexão, sem geração de método Main.

B)

A compilação é viável sem using System; porque diretivas using implícitas já colocam System no escopo; o compilador sintetiza um método de ponto de entrada e uma estrutura de suporte para encapsular as instruções de nível superior.

C)

A compilação é viável sem using System; porque Console pertence ao namespace global; não ocorre geração de método de ponto de entrada, pois instruções de nível superior são executadas diretamente pelo runtime.

D)

A compilação é viável, mas apenas se o arquivo declarar um namespace e uma classe Program; as instruções de nível superior servem apenas para scripts e não geram executáveis.

E)

A compilação é viável sem using System; porque o compilador converte Console.WriteLine em System.Console.WriteLine no IL; contudo, a ausência de ponto de entrada exige que o projeto defina StartupObject no .csproj.

Exercício 2:

Você está estruturando uma solução com múltiplos projetos (src/, tests/ etc.) e quer atingir simultaneamente três objetivos: (1) manter o grafo de dependências entre módulos explícito e integrado ao build, (2) unificar versões de pacotes externos para evitar divergências entre projetos, e (3) aplicar políticas transversais (por exemplo, nulabilidade e usings implícitos) de forma consistente. Qual alternativa descreve uma configuração coerente com esses objetivos?

A)

Declarar todas as dependências internas como referências a DLLs já compiladas e registrar as versões de pacotes no arquivo .sln, pois ele é o contêiner que controla compilação e dependências; aplicar regras transversais diretamente em launchSettings.json.

B)

Usar ProjectReference para dependências internas entre .csproj, centralizar versões em Directory.Packages.props na raiz (habilitando ManagePackageVersionsCentrally e declarando PackageVersion), deixar os projetos com PackageReference sem Version, e aplicar políticas comuns via Directory.Build.props e regras de estilo via .editorconfig.

C)

Evitar ProjectReference para reduzir acoplamento, preferindo copiar código entre projetos; centralizar versões em Directory.Build.targets; e ativar nulabilidade apenas nos projetos de testes para reduzir avisos.

D)

Manter ProjectReference, mas registrar versões diretamente em cada .csproj para tornar os projetos independentes; aplicar políticas comuns duplicando os mesmos <PropertyGroup> em cada projeto para evitar arquivos na raiz.

E)

Colocar TargetFrameworks em Directory.Packages.props para que o multi-targeting seja herdado pelos projetos; e registrar PackageReference com Version apenas no projeto da API, pois os demais herdam por transitividade.

Exercício 3:

Em C# 14, em um programa com instruções de nível superior (top-level statements), considere a linha cliente?.Total += 100m;, em que cliente tem o tipo Cliente? e Total é uma propriedade decimal com get; set;. Qual alternativa descreve corretamente o comportamento dessa linha, incluindo o caso em que cliente seja null?

A)

A expressão sempre executa a soma e grava 100m em Total, pois o operador ?. apenas afeta a leitura posterior.

B)

A expressão sempre lança NullReferenceException quando cliente é null, pois += exige leitura e escrita e ignora o ?..

C)

A expressão não executa nenhuma leitura nem escrita se cliente for null; se cliente não for null, ela lê Total, soma 100m e grava o resultado de volta em Total.

D)

A expressão só compila se Total for do tipo decimal?, pois o operador ?. torna o lado esquerdo anulável e impede += com decimal.

E)

A expressão compila, mas, quando cliente é null, ela grava 0m em Total por "valor-padrão" do tipo decimal.

Exercício 4:

Considere o tipo public record Produto(int Id, string Nome, decimal Preco); e o uso: var p2 = p1 with { Preco = 18m };. Qual alternativa está correta sobre a identidade das instâncias, a igualdade e o efeito do with, assumindo que p1 foi criado como new Produto(1, "Café", 15m)?

A)

A expressão with altera p1 no lugar e devolve uma referência para a mesma instância, de modo que p1.Preco passa a ser 18m.

B)

A expressão with cria uma nova instância, preserva p1 inalterado e faz com que p1 e p2 sejam iguais por referência, porque records usam identidade de objeto.

C)

A expressão with cria uma nova instância, preserva p1 inalterado e faz com que p1 e p2 sejam diferentes em igualdade estrutural, pois diferem em Preco.

D)

A expressão with só é válida se o record tiver campos mutáveis com set;, pois ela depende de atribuições posteriores para clonar o objeto.

E)

A expressão with cria uma nova instância, mas o compilador torna Id e Nome indefinidos (default) no clone, a menos que eles sejam repetidos no inicializador.

Exercício 5:

Você quer um projeto de console em estilo SDK (SDK-style) voltado a net10.0, com referências anuláveis habilitadas, ImplicitUsings habilitado e sem geração automática de atributos de assembly (ou seja, com controle manual via código, quando desejado). Qual arquivo .csproj atende corretamente a esses requisitos com a configuração essencial, sem depender de listagem manual de arquivos .cs?

A)

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <GenerateAssemblyInfo>>false</GenerateAssemblyInfo>
  </PropertyGroup>
</Project>
```

B)

```
<Project>
  <PropertyGroup>
    <TargetFrameworkVersion>v10.0</TargetFrameworkVersion>
    <OutputType>Console</OutputType>
    <Nullable>enabled</Nullable>
    <ImplicitUsings>true</ImplicitUsings>
    <GenerateAssemblyInfo>>false</GenerateAssemblyInfo>
  </PropertyGroup>
</Project>
```

C)

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
  <ItemGroup>
    <Compile Include="Program.cs" />
  </ItemGroup>
</Project>
```

D)

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>disable</ImplicitUsings>
    <GenerateAssemblyInfo>>false</GenerateAssemblyInfo>
  </PropertyGroup>
</Project>
```

E)

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Library</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <LangVersion>14.0</LangVersion>
    <Nullable>disable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <GenerateAssemblyInfo>true</GenerateAssemblyInfo>
  </PropertyGroup>
</Project>
```

Exercício 6:

Em um trecho de código em C#, considere a sequência abaixo:

```
using System;
using System.Collections.Generic;

class Programa
{
    static void Main()
    {
        List<double> constants = [ Math.PI, Math.E ];
        System.Console.WriteLine(constants.Count);
    }
}
```

Qual é o papel do ponto nas diferentes ocorrências?

A)

O ponto é usado somente para acessar membros de instância; em Math.PI e em Math.E, ele funciona como acesso à instância, e não há relação com nomes qualificados de namespaces ou de tipos.

B)

O ponto forma nomes qualificados em System.Console e em System.Collections.Generic, e também realiza acesso a membros; em Math.PI e em Math.E, o acesso é a membros estáticos, e, em constants.Count, o acesso é a membro de instância; além disso, expressões do tipo e.M e invocações do tipo e.M(e1, ..., ev) integram o conjunto de expressões de acesso a membro que o compilador precisa vincular a um membro concreto durante a resolução.

C)

O ponto forma nomes qualificados somente quando aparece após using; fora disso, ele deixa de ser operador e vira apenas parte do identificador, razão pela qual System.Console.WriteLine não participa do mesmo mecanismo de acesso a membro de constants.Count.

D)

O ponto pode formar nomes qualificados de namespaces e de tipos, mas não é usado para acessar membros de tipos; em Console.WriteLine e em constants.Count, a seleção do membro ocorre por um mecanismo independente do ponto.

E)

O ponto é um operador que funciona para acesso a membros de instância, porém membros estáticos como Math.PI e Console.WriteLine não usam o ponto como operador de acesso; nessas formas, o ponto atua apenas como separador sintático sem impacto na vinculação pelo compilador.

Exercício 7:

Você precisa redesenhar uma classe de conta bancária em C# com as seguintes exigências simultâneas: (i) o código externo não pode alterar o saldo de maneira arbitrária; (ii) depósitos com quantia não positiva devem falhar; (iii) saques com quantia não positiva ou acima do saldo devem falhar; (iv) o saldo deve poder ser consultado externamente; (v) a solução deve reduzir o risco de estados inválidos e evitar dependências do código cliente em detalhes internos. Qual das alternativas abaixo corresponde ao código que melhor atende a essas exigências?

A)

```
public class ContaBancaria
{
    public decimal Saldo;

    public bool Depositar(decimal quantia)
    {
        Saldo += quantia;
        return true;
    }

    public bool Sacar(decimal quantia)
    {
        Saldo -= quantia;
        return true;
    }
}
```

```
}
```

B)

```
public class ContaBancaria
{
    public decimal Saldo { get; set; }

    public bool Depositar(decimal quantia)
    {
        if (quantia <= 0) return false;
        Saldo += quantia;
        return true;
    }

    public bool Sacar(decimal quantia)
    {
        if (quantia <= 0) return false;
        Saldo -= quantia;
        return true;
    }
}
```

C)

```
public class ContaBancaria
{
    private decimal saldo;

    public decimal Saldo
    {
        get { return saldo; }
        private set { saldo = value; }
    }

    public ContaBancaria(decimal saldoInicial = 0m)
    {
        if (saldoInicial < 0m)
            throw new ArgumentOutOfRangeException(nameof(saldoInicial));

        saldo = saldoInicial;
    }

    public bool Depositar(decimal quantia)
    {
        if (quantia <= 0)
            return false;
        saldo += quantia;
        return true;
    }
}
```



```
public bool Sacar(decimal quantia)
{
    if (quantia <= 0 || quantia > saldo)
        return false;
    saldo -= quantia;
    return true;
}
```

D)

```
public class ContaBancaria
{
    private decimal saldo;

    public decimal Saldo
    {
        get { return saldo; }
        set { saldo = value; }
    }

    public bool Depositar(decimal quantia)
    {
        if (quantia <= 0) return false;
        saldo += quantia;
        return true;
    }

    public bool Sacar(decimal quantia)
    {
        if (quantia <= 0 || quantia > saldo) return false;
        saldo -= quantia;
        return true;
    }
}
```

E)

```
public class ContaBancaria
{
    private decimal saldo;

    public decimal Saldo
    {
        get { return saldo; }
    }

    public bool Depositar(decimal quantia)
    {
        saldo += quantia;
        return true;
    }
}
```

```
public bool Sacar(decimal quantia)
{
    saldo -= quantia;
    return true;
}
```

Exercício 8:

Em uma classe Pessoa que encapsula a idade, decide-se alterar a representação interna de "anos" para "meses", mantendo inalterado o contrato público de Idade (continuar expondo anos) e preservando o invariante $\text{Idade} \geq 0$. Qual alternativa descreve a refatoração que melhor preserva o encapsulamento, mantém o invariante centralizado e reduz o risco de inconsistências internas?

A)

A substituição do campo privado por um campo público `idadeEmMeses`, mantendo a propriedade `Idade` apenas como um atalho conveniente de leitura.

B)

A manutenção do campo privado e a alteração da propriedade `Idade` para retornar meses, contando que a documentação externa seja atualizada.

C)

A substituição do campo privado por `private int idadeEmMeses;`, com a propriedade `Idade` convertendo $\text{anos} \leftrightarrow \text{meses}$, validando no `set` e armazenando sempre meses no campo privado; além disso, os métodos internos que alteram a idade passam a fazê-lo por meio da propriedade (quando a lógica da propriedade for relevante).

D)

A adoção de uma autopropriedade `public int Idade { get; set; }`, com a validação do invariante transferida para o código cliente antes de cada atribuição.

E)

A remoção do `set` público e a exposição de um método `AtualizarIdade(int anos)` que atribui diretamente no campo privado, sem reutilizar a lógica da propriedade.

Exercício 9:

Considere uma classe que impõe o invariante idade ≥ 0 por meio de um campo privado e de uma propriedade Idade com validação no set. Suponha que um método público precise recalcular a idade a partir de múltiplos parâmetros e possa, durante o cálculo, obter valores intermediários negativos. À luz da noção de invariante (válido antes e depois de chamadas a métodos públicos, podendo ser quebrado temporariamente apenas durante a execução interna), qual alternativa descreve uma implementação que respeita o invariante “em todos os momentos observáveis externamente”?

A)

O método atribui idade = -1 diretamente no campo privado, chama um callback fornecido pelo código cliente para “notificar o progresso” e só então restaura a idade para um valor não negativo antes de retornar.

B)

O método atribui idade = -1 diretamente no campo privado e, em seguida, executa operações que podem lançar exceções; se ocorrer exceção, o método termina sem garantir a restauração.

C)

O método mantém todos os valores intermediários em variáveis locais, valida o valor final e realiza uma única atribuição ao estado do objeto (via propriedade, quando a lógica de validação for parte do contrato), assegurando que nenhuma interação externa aconteça enquanto o valor intermediário estiver inválido.

D)

O método usa a propriedade Idade para primeiro atribuir um valor negativo (como sentinela) e depois atribuir o valor correto, contando que, “no final”, o estado ficará consistente.

E)

O método expõe uma referência direta ao campo interno (por exemplo, retornando um acesso mutável indireto) para que o código cliente finalize o ajuste e então “conserte” a idade.

Exercício 10:

Analise o código a seguir. Considere que ele está distribuído em dois arquivos na mesma solução:

Arquivo Pessoa.cs:

```
public record Pessoa(string Nome, int Idade);
```

Arquivo Program.cs (com instruções de nível superior):

```
var p = new Pessoa("Ana", 30);  
var p2 = p with { Idade = 31 };  
var p3 = p with { Idade = 30 };
```

Qual alternativa está correta sobre (i) o que pode ou não compilar envolvendo `init`, (ii) o efeito de `with` e (iii) a semântica de igualdade de `record`?

A)

A atribuição `p.Idade = 31`; compila e altera `p`, porque `record` é apenas uma "classe com sintaxe curta"; já `with` retorna a mesma instância e altera o objeto original.

B)

A expressão `with` cria um novo objeto, mas `Equals` permanece referencial; portanto, `p3.Equals(p)` é falso, mesmo com os mesmos valores, a menos que `Equals` seja sobrescrito manualmente.

C)

A atribuição `p.Idade = 31`; falha em compilação; `p2` e `p3` são novas instâncias; além disso, `p3.Equals(p)` retorna `true`, embora `object.ReferenceEquals(p3, p)` retorne `false`.

D)

É proibido declarar qualquer membro mutável em `record`; portanto, mesmo que se escreva uma propriedade com `set`, o compilador rejeita por violar a definição de `record`.

E)

A igualdade em `record` compara apenas o primeiro componente posicional (`Nome`), por isso `p2.Equals(p)` seria `true` se `Nome` for igual, ainda que `Idade` difira.

